

**Implementation of Snort IDS and Detecting Brute
Force Attack on SSH and FTP**

BY

SANJITH S

Abstract

This project represents the implementation, and testing of a Snort-based Intrusion Detection System (IDS) in a Linux server environment for monitoring network traffic and detecting suspicious activities targeting network services. The primary objective of the project is to implement a network intrusion detection system to identify brute-force attacks against commonly exposed services and generating security logs for analysis. Snort 3 is installed and configured as the network IDS, with custom detection rules developed to identify specific patterns of malicious or abnormal network activity.

As part of the implementation, Secure Shell (SSH) and File Transfer Protocol (FTP) services are configured on the Linux server. SSH is configured for remote administration, while an FTP server using vsftpd is deployed for file transfer operations. Default port is changed from port 22 to a non-default port as an additional hardening measures. Snort is configured with supporting components, traffic monitoring settings, and custom rules to analyse packets associated with the configured services. The configuration is validated to ensure that Snort can successfully process network traffic and generate alerts.

To evaluate IDS, controlled brute-force attack scenarios are stimulated against both the SSH service and the FTP service. Repeated authentication attempts are generated using Hydra tool, stimulating a real-world attack. The network traffic is captured and inspected by Snort, while each activity is being recorded in the log. Custom Snort rules are used to identify repeated connection attempt towards the services, and alerts are generated when the configured detection conditions were matched.

This testing demonstrated that the implemented system can successfully monitor the network traffic, detect any suspicious login attempts associated against both services, and generate corresponding security alerts. The generated logs can be used as evidence for detecting suspicious activity. Therefore, this project is demonstrated the practical application of Snort-based IDS, for monitoring, detecting intrusions, generating alerts and event logging. Also, demonstrating the importance of security the exposed services. Overall, the project provides understanding of Snort, IDS detection rules, alerting and security event logging.

Introduction

The increasing dependence on network-based services has made the security of servers and network infrastructure an important aspect of modern information security. Secure Shell (SSH) and File Transfer Protocol (FTP) are commonly deployed to provide remote administration and file transfer capabilities. When these services are exposed to a network, they can become targets for unauthorized access attempts, password guessing, and brute-force attacks. Continuous monitoring of network traffic is necessary to identify suspicious activities at an early stage and provide security to those services.

An Intrusion Detection System (IDS) is a security mechanism designed to monitor network or system activity and identify events that may indicate malicious or unauthorized behaviour. A network-based IDS analyses packets transmitted across the network and compares observed traffic against predefined detection rules. Upon detecting the suspicious activity, IDS can generate an alert and record relevant information in security logs. These logs can be used for security analysis, troubleshooting, incident investigation, and as evidence of monitoring attack activity.

In this project, Snort 3 is implemented as a network-based Intrusion Detection System in a Linux server environment. Snort is configured to monitor network traffic passing through the selected network interface and inspect traffic through the services deployed on the server. Custom rules are configured to identify traffic patterns and repeated connection attempts that may indicate brute-force activity.

To provide a controlled environment for evaluating the IDS, two network services, SSH and FTP, are configured on the Linux server. SSH is used to provide secure remote administration of the server. The default SSH port TCP 22 is changed to a different non-default port. This demonstrates an additional security measure and allows the IDS to monitor a service operating on a non-standard port. Also, server configuration is done to ensure that SSH work perfectly in the modified port. An FTP server using vsftpd is also configured and provides a separate service through which network traffic and authentication attempts can be observed. The FTP service operates on the standard TCP port 21, allowing the project to compare the monitoring of a service operating on its conventional port with the monitoring of SSH on a modified port.

The detection capability of the Snort IDS is evaluated through controlled brute-force attack simulations. Repeated authentication attempts are generated against the SSH and FTP services using Hydra within the environment. These simulated attacks represent a common threat in which an attacker repeatedly attempts different username and password combinations to gain unauthorized access. During testing, Snort monitors the resulting network traffic and applies the configured detection rules to identify repeated connection or authentication activity. When a suspicious activity is detected, Snort generates an alert that can be stored in the log files. These logs provide evidence of detected activity and can be analyzed to understand the nature of the attack, the targeted services.

Objective

The objective of this projects are

- To install and configure Snort 3 on a Linux server and establish it as a network-based Intrusion Detection System.
- To configure SSH and FTP services on the Linux server to create controlled and realistic network service environments for security testing.
- To change the SSH port to a non-default port and configuring the settings to create a secured remote access.
- To configure an FTP server using vsftpd on standard port and ensuring its availability and security-testing activities.
- To configure custom Snort detection rule to identify suspicious connection and repeated access attempt targeting both SSH and FTP services.
- To simulate brute-force attacks against both SSH and FTP services using Hydra in the testing environment.
- To evaluate Snort's detection by monitoring the network traffic generated during the brute-force attack and verifying that the configured rule is detecting the attack.
- To configure Snort alerting and logging mechanisms so that detected security events are recorded for further investigation.
- To perform a practical demonstration of network-based IDS.

System Requirements

- Ubuntu server 24.04 LTS – testing environment to deploy Snort
- Snort 3.12.3 – serves as the main Intrusion Detection System
- OpenSSH Server - Provides the SSH remote-access service used as one of the targets for security testing.
- Vsftpd - Provides the FTP service used as the second target for security testing. It allows FTP authentication traffic to be generated and monitored by Snort.
- Hydra - Used to perform controlled brute-force attack simulations against the SSH and FTP services.

This combination of software provides all the components required to deploy the IDS, configure the target services, generate controlled attacks, monitor network traffic, detect suspicious activity, and record the security events.

Working Principle of Snort IDS

Snort operates as a network-based Intrusion Detection System by continuously monitoring and analyzing network traffic passing through a configured network interface. Its function is to identify network packets that match predefined patterns or conditions associated with suspicious activities.

In this implementation, Snort is configured to monitor traffic directed toward the SSH and FTP services and to identify repeated connection attempts associated with brute-force attacks.

The working process begins when network packets arrive at the network interface monitored by Snort. Snort uses its packet acquisition mechanism to capture the traffic from the selected interface. After capturing the packets, Snort processes the network traffic through its packet decoding and inspection mechanisms. This allows Snort to understand the structure and characteristics of the traffic before applying its detection rules. The processed traffic is then evaluated against the configured Snort detection rules. These rules are customized to evaluate the traffic and generate alert when the specified conditions are satisfied.

When the traffic satisfies the conditions defined by a rule, Snort classifies the activity as a security event and generates an alert. The alert contains information such as the alert message, source and destination addresses, ports and the rule identifier.

Installation and Configuration

- Update the Linux environment
sudo apt update
sudo apt upgrade -y

Installing the Package

- sudo apt install -y \
build-essential \
cmake \
git \
pkg-config \
libpcap-dev \
libpcr2-dev \
libdumbnet-dev \
libnet1-dev \
zlib1g-dev \
liblzma-dev \
libssl-dev \
liblua5.1-dev \
libhwloc-dev \
libmnl-dev \
libunwind-dev \
bison \
flex \
autoconf \
libtool \
libcmocka-dev \
libsqlite3-dev \
uuid-dev \
libhyperscan-dev \
wget
- Installing and configuring DAQ
cd /usr/src
sudo git clone <https://github.com/snort3/libdaq.git>
- Compiling DAQ
cd libdaq
sudo ./bootstrap
sudo ./configure
sudo make -j\$(nproc)
sudo make install

- Downloading Snort
`cd /usr/src`
`sudo git clone https://github.com/snort3/snort3.git`
- Building Snort
`cd snort3`
`sudo ./configure_cmake.sh --prefix=/usr/local --enable-tcmalloc`
`cd build`
`sudo make -j$(nproc)`
`sudo make install`
- Refresh the library
`sudo ldconfig`

Create Snort Directories

- Create directories for snort
`sudo mkdir -p /usr/local/etc/snort/rules`
`sudo mkdir -p /usr/local/etc/snort/so_rules`
`sudo mkdir -p /usr/local/etc/snort/lists`
`sudo mkdir -p /var/log/snort`
`sudo mkdir -p /usr/local/etc/snort/plugins`
- Copying the configuration files
`cd /usr/src/snort3`
`sudo cp etc/*.lua /usr/local/etc/snort/`
`sudo cp -r lua /usr/local/etc/snort/`

Changing the default SSH Port

- Open the ssh configuration file
`sudo nano /etc/ssh/sshd_config`

Locate the line that configures the port and change to your non-default port,

`#port=22`

Change it to

`Port=2222`

Save it using `ctrl+o`, enter it and exit using `ctrl+x`

- Restart the ssh service
`sudo systemctl restart ssh`
`sudo systemctl restart ssh.service`

Create the Brute Force Detection Rule for SSH

- Open the custom file to configure the rule
`sudo nano /usr/local/etc/snort/rules/local.rules`

Paste the rule below into the file, save it ctrl+o, enter it and exit ctrl+x.

```
alert tcp any any -> any 2222(  
  msg:"SSH Brute Force Attempt";  
  flow:to_server;  
  flags:S;  
  detection_filter:track by_src, count 5, seconds 60;  
  sid:1000001;  
  rev:1;  
)
```

Creating the FTP Server

- Installing Ftp server
`sudo apt install vsftpd -y`
`sudo systemctl enable vsftpd`
- Configuring backup file
`sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.backup`
- Configuring the ftp server
`sudo nano /etc/vsftpd.conf`

Add and change the configurations mentioned below

`listen=YES`

`listen_ipv6=NO`

`anonymous_enable=NO`

`local_enable=YES`

`write_enable=YES`

`local_umask=022`

`dirmessage_enable=YES`

`use_localtime=YES`

`xferlog_enable=YES`

`connect_from_port_20=YES`

```
xferlog_std_format=NO  
  
log_ftp_protocol=YES  
  
dual_log_enable=YES  
  
chroot_local_user=YES  
  
allow_writeable_chroot=YES  
  
pam_service_name=vsftpd  
  
secure_chroot_dir=/var/run/vsftpd/empty  
  
user_sub_token=$USER  
  
local_root=/home/$USER/ftp  
  
pasv_enable=YES  
  
pasv_min_port=40000  
  
pasv_max_port=40100
```

Save the file `ctrl+o`, enter and exit using `ctrl+x`

Creating the FTP Directories

- Create a user.
`sudo adduser ftpmain`

create the user and password.

- Create the user directory.
`sudo mkdir -p /home/ftpuser/ftp/files`
- Assign the ownership to the user
`sudo chown nobody:nogroup /home/ftpmain/ftp`
`sudo chmod a-w /home/ftpmain/ftp`
`sudo chown ftpmain:ftpmain /home/ftpmain/ftp/files`

Test the FTP Login

From another Terminal,

```
ftp <server-ip>  
username:ftpmain  
password:*****
```

Create the Brute Force Detection Rule for FTP

- Open the local rule file to configure the rule
`sudo nano /usr/local/etc/snort/rules/local.rules`
- Add the following rule

```
alert tcp any any -> any 21 (  
  msg:"FTP Brute Force Attack Detected";  
  flags:S;  
  flow:stateless;  
  detection_filter:track by_src,count 5,seconds 60;  
  sid:1000002;  
  rev:1;  
)
```

Save it and exit.

Configure the snort.lua file

- Open the configuration file
`sudo nano /usr/local/etc/snort/snort.lua`
- Locate the 'ips' section and include the local rule file.

```
ips =  
{  
  rules = [[  
    include /usr/local/etc/snort/rules/local.rules  
  ]]  
}
```
- Locate the log file setting and enable the log file.

```
alert_fast = {  
  file = true  
}
```

Save the file and exit.

- Run the snort.
`sudo snort -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/snort/rules/local.rules -i enp0s3`

Generate a Brute force attack

From another machine, initiate the brute force attack on ssh service,

```
hydra -l username -P /usr/share/wordlists/rockyou.txt ssh://<server-ip:port>
```

After that, initiate an attack on ftp service,

```
hydra -l username -P /usr/share/wordlists/rockyou.txt ftp://<server-ip>
```

Advantages of Snort IDS

- **Open Source and Free** - Snort is open-source software, making it freely available for deployment and can be modified according to projects and organizational purposes
- **Customizable Detection Rules** - Administrators can create custom rules to detect specific attack patterns, protocols, ports, or connection behaviours.
- **Supports Multiple Protocols** - Snort can inspect traffic associated with different network and application protocols, making it useful for monitoring services such as SSH, FTP, HTTP.
- **Alert Generation and Logging** - When traffic matches a configured detection rule, Snort can generate security alerts and record the events in log files.
- **Useful for Security Testing and Learning** - Snort provides a practical environment for understanding network traffic analysis, IDS rule development, attack detection, and security event logging.

Disadvantages of Snort IDS

- **Requires Proper Configuration** - Snort's effectiveness depends heavily on its configuration and detection rules. Incorrect rules, interfaces, ports, or inspection settings can result in missed alerts.
- **Limited Detection of Unknown Attacks** - Traditional Snort rule-based detection primarily relies on known patterns and predefined conditions. New attacks do not match the defined rule and does not detect the attack
- **Encrypted Traffic Limitation** - Snort cannot directly inspect the contents of properly encrypted traffic without access to the decrypted data.
- **Requires Regular Rule Maintenance** - Detection rules need to be updated and tuned as new threats emerge and network environments change.
- **Performance Impact on High-Traffic Networks** - Processing and inspecting large volumes of network traffic can consume significant CPU and memory resources. Extensive rule sets can further increase the processing requirements.
- **Requires technical expertise** - Administrators need knowledge of networking, protocols, Snort configuration, and rule syntax to create effective detection policies and correctly interpret generated alerts.

Conclusion

This project successfully demonstrated the implementation and practical use of Snort 3 as a Network Intrusion Detection System (NIDS) in a Linux-based server environment. The project combined network monitoring, service configuration, attack simulation, intrusion detection, and security event logging to create a controlled environment for evaluating the effectiveness of an IDS.

As part of the implementation, SSH and FTP services were configured on the Linux server and monitored by Snort. The default SSH port was changed to a non-default port as an additional security-hardening measure, while the FTP service was configured using vsftpd on TCP port 21. Snort was configured to monitor the appropriate network interface, inspect network traffic, and apply custom detection rules to identify suspicious connection patterns.

Brute-force attacks were performed against both SSH and FTP services using Hydra. These tests demonstrated how repeated authentication attempts generate identifiable network traffic that can be monitored by an IDS. Snort successfully processed the network traffic and generated security alerts.

Snort logs, together with the corresponding SSH and FTP service logs, can be used to correlate network-level alerts with service-level authentication activity. This provides a better understanding of the attack.

Overall, the project achieved its objective of demonstrating practical experience in configuring network services, developing detection rules, analyzing network traffic, and validating IDS behaviour through controlled attack scenarios. The implementation highlights the importance of combining secure service configuration with continuous network monitoring to improve the visibility and security of networked systems.